If you want to change the variable whatever the initial value is (perhaps with an updated value that depends on the current value), the update of `expected` becomes useful; each time through the loop, `expected` is reloaded, so if no other thread modifies the value in the meantime, the `compare_exchange_weak()` or `compare_exchange_strong()` call should be successful the next time around the loop. If the calculation of the value to be stored is simple, it may be beneficial to use `compare_exchange_weak()` in order to avoid a double loop on platforms where `compare_exchange_weak()` *can* fail spuriously (and so `compare_exchange_strong()` contains a loop). On the other hand, if the calculation of the value to be stored is itself time consuming, it may make sense to use `compare_exchange_strong()` to avoid having to recalculate the value to store when the `expected` value hasn't changed. For `std::atomic<bool>` this isn't so important—there are only two possible values after all—but for the larger atomic types this can make a difference.

The compare/exchange functions are also unusual in that they can take *two* memory-ordering parameters. This allows for the memory-ordering semantics to differ in the case of success and failure; it might be desirable for a successful call to have `memory_order_acq_rel` semantics whereas a failed call has `memory_order_relaxed` semantics. A failed compare/exchange doesn't do a store, so it can't have `memory_order_release` or `memory_order_acq_rel` semantics. It's therefore not permitted to supply these values as the ordering for failure. You also can't supply stricter memory ordering for failure than for success; if you want `memory_order_acquire` or `memory_order_seq_cst` semantics for failure, you must specify those for success as well.

If you don't specify an ordering for failure, it's assumed to be the same as that for success, except that the `release` part of the ordering is stripped: `memory_order_release` becomes `memory_order_relaxed`, and `memory_order_acq_rel` becomes `memory_order_acquire`. If you specify neither, they default to `memory_order_seq_cst` as usual, which provides the full sequential ordering for both success and failure. The following two calls to `compare_exchange_weak()` are equivalent:

```
std::atomic<bool> b;
bool expected;
b.compare_exchange_weak(expected,true,
    memory_order_acq_rel,memory_order_acquire);
b.compare_exchange_weak(expected,true,memory_order_acq_rel);
```

I'll leave the consequences of the choice of memory ordering to section 5.3.

One further difference between `std::atomic<bool>` and `std::atomic_flag` is that `std::atomic<bool>` may not be lock-free; the implementation may have to acquire a mutex internally in order to ensure the atomicity of the operations. For the rare case when this matters, you can use the `is_lock_free()` member function to check whether operations on `std::atomic<bool>` are lock-free. This is another feature common to all atomic types other than `std::atomic_flag`.

The next-simplest of the atomic types are the atomic pointer specializations `std::atomic<T*>`, so we'll look at those next.